claude-windows / 56b7f849-5d56-4770-93e0-b3b4da943700

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\56b7f849-5d56-4770-93e0-b3b4da943700\subagents\workflows\wf_49729ae7-962\agent-aa63b11884360ba33.jsonl`
- SHA-256: `9b4b617391924a2865c1a987805bdda5a628f46ef65d4309538c4d32b7a1a870`
- Source modified: `2026-06-07T08:14:14+00:00`
- Imported at: `2026-07-05T16:48:27+00:00`
- project: `wf_49729ae7-962`
- session_id: `56b7f849-5d56-4770-93e0-b3b4da943700`

Transcript

1. user (2026-06-07T08:13:58.708Z)

You are a USER of a sports-video tool who has read ONLY its README/FAQ ? you have NEVER seen the source code. A processing run looks wrong and the tool wrote a per-video report. Read EXACTLY these two files and NOTHING else (do not open source code or any other file):
1. The README/FAQ: C:\Users\avidu\Projects\badminton-highlight-indexer\README.md
2. The tool's technical report for the suspicious run: C:\Users\avidu\AppData\Local\Temp\obseval\reports\indexer_no_api_key.report.md

Using ONLY those two, tell me: what went wrong, exactly what you'd do to fix it, and which FAQ section (by heading text or Q-number) you used. Be honest about whether you could truly self-debug this from the report alone, and list anything in the report that was confusing, ambiguous, or under-explained.

2. assistant (2026-06-07T08:13:59.897Z)

I'll read those two files to diagnose the issue.

3. user (2026-06-07T08:14:00.118Z)

1 # Badminton Highlight & Rally Indexer
2
3 A local, AI-powered web application and CLI utility designed to process long-form badminton recordings (e.g., from GoPro), run comprehensive pluggable validations (quality, integrity, and relevance checks), index all rallies into a relational SQLite database, and dynamically compile custom highlight reels using high-speed FFmpeg stitching.
4
5 The pipeline is built around three pluggable registries ? **validators**, **segmenters**, and **AI providers** ? plus a **sport profile** registry so the same engine can target other racket/court sports in the future. Badminton is the default profile.
6
7 > **Where this is heading:** the project is evolving from an AI-API-dependent prototype into a self-improving tool that learns rally detection *offline* from a commercially-licensed, annotated corpus. See **docs/ROADMAP.md** for the four-phase plan and **docs/DECISIONS.md** (ADR-002?004) for the rationale.
8
9 ---
10
11 ## ? Key Features
12
13 * **Pluggable Video Validation Framework**: Easily extendable base-class architecture that automatically runs checks on:
14 * *Static Format & Resolution*: Verifies container is MP4, resolution >= 720p, and framerate >= 30 FPS.
15 * *PTS Timeline Continuity*: Scans keyframe timestamps for timeline gaps, jumps, or editing splices to detect tampering or splices.

```
16          *   *Scene Cut Density Check*: Measures sub-sampled HSV frame histogram correlation
to verify visual cuts. A continuous raw camera stream should have near 0 cuts.
17          *   *Blurriness Check*: Calculates average frame Laplacian variance to reject out-
of-focus footage.
18          *   *Relevance (Is it Badminton?)*: Uses high-performance HSV color court matching
to identify court layouts.
19      *   **Three Pluggable Rally Segmenters** (`backend/indexer/`):
20          *   `yolo_hybrid` **(default)** ? two-stage pipeline. Stage 1 runs **torchvision
person detection** (Faster R-CNN, BSD-licensed) with **ByteTrack** tracking (supervision, MIT-
licensed) to cheaply filter the video down to a small set of high-motion candidate windows.
Stage 2 hands each candidate (with a small padding) to the AI provider for high-precision
semantic boundaries and shot counts. Massively cheaper than running the LLM on the full video.
*(The registry key remains `yolo_hybrid` for back-compat; the AGPL-licensed YOLOv8 was replaced
with permissive components ? see [docs/MONETIZATION_AUDIT.md](docs/MONETIZATION_AUDIT.md) and
ADR-005.)*
21          *   `gemini` ? sends the entire video (or a `chunk_duration` slice) to Google Gemini
directly. Highest precision per call, highest cost.
22          *   `motion` ? zero-dependency OpenCV pixel-difference heuristic. No API key
required. Useful as a baseline or for offline/air-gapped use.
23      *   **Pluggable AI Provider Registry** (`backend/providers/`): The Gemini provider ships
in-tree; new providers (e.g. an OpenAI or local-model backend) can be registered without
touching the segmenters.
24      *   **Pluggable Sport Profile Registry** (`backend/sports/`): The per-sport LLM prompt
and relevance config live in a `SportProfile`. Add a new sport by subclassing `SportProfile` and
registering it.
25      *   **SQLite Indexing**: Each processed video is keyed by MD5 hash. Play segments
(start/end/duration/server/receiver/metadata) and per-segment events are persisted in
`output/sports_indexer.db`. Re-running on the same file is a cache hit.
26      *   **Query System**: Filter indexed play segments by `server`, `receiver`,
`duration_min`, and `event_type` (the schema supports rich event types like
smashes/serves/fouls; today the shipped segmenters mainly populate timing + estimated
`shot_count`).
27      *   **Zero-Dependency Local UI**: Served statically directly by FastAPI (Vite + Node.js
NOT required!) featuring a glassmorphic dashboard, responsive filtering, and immediate clip
stitching.
28      *   **High-Speed FFmpeg Stitching** with configurable `begin_padding`, `end_padding`,
and `min_shot_count` filtering.
29      *   **Detailed CLI Debugging Tools**: `--debug-clip <t>` inspects frame metrics,
validator scores, and motion values around a timestamp. `--trim-start/--trim-end` extracts an
arbitrary segment via stream-copy ffmpeg for fast iteration.
30
31      ---
32
33      ## ?? Setup & Diagnostics
34
35      ### 1. External System Requirements
36      The indexer depends on **FFmpeg** and **FFprobe** to parse media containers, extract
keyframes, and stitch clips.
37
38      *   **On Windows (PowerShell as Admin)**:
39          ```powershell
40          winget install Gyan.FFmpeg
41          ```
42          *Restart your terminal afterwards to register the path changes.*
43
44      ### 2. Auto-Installer & Diagnostics Checker
45      Run the diagnostics checker script in the root directory to verify your dependencies,
initialize default configs, and install PIP packages:
46      ```bash
47      python setup.py
48      ```
49      This script will:
50      *   Validate `ffmpeg` and `ffprobe` are present in your PATH.
51      *   Initialize a default `config.json` (if one does not exist) with thresholds for blur,
court HSV bounds, motion signals, YOLO velocity, chunking, and stitching.
```

```
52      *   Detect whether an NVIDIA GPU is available (via `nvidia-smi`) and select the matching
PyTorch wheel index automatically (`cu124` if GPU present, `cpu` otherwise).
53      *   Install everything from `requirements.txt`: `fastapi`, `uvicorn`, `opencv-python`,
`numpy`, `pydantic`, `jinja2`, `python-dotenv`, `google-genai`, `torch`, `torchvision`,
`supervision`.
54
55      > The default `yolo_hybrid` segmenter uses **torchvision**'s COCO-pretrained Faster
R-CNN, whose weights are downloaded and cached automatically by torchvision on first run ?
nothing is bundled in the repo. (Pick a lighter/heavier detector via `indexing.detector_model`:
`fasterrcnn_resnet50_fpn` (default), `fasterrcnn_mobilenet_v3_large_fpn`, or
`retinanet_resnet50_fpn`.)
56
57      ---
58
59      ## ? Running the Application
60
61      ### 1. Launching the Local Web App Dashboard
62      To start the FastAPI server and open the web dashboard:
63      ```bash
64      python -m backend.main --server --port 8000
65      ```
66      Open your browser and navigate to **[http://localhost:8000](http://localhost:8000)**.
67      *Simply input your local raw video file path in the sidebar to process, validate, index,
and compile highlights instantly.*
68
69      ### 2. Running in CLI Processing Mode
70      To process and index a video completely via terminal commands:
71      ```bash
72      python -m backend.main --video-path "C:/path/to/my_video.mp4"
73      ```
74      This outputs a validation summary and indexes all rallies directly into the local SQLite
database, using whichever segmenter is set as `default_segmenter` in `config.json`
(`yolo_hybrid` by default).
75
76      To explicitly pick a segmenter for one run:
77      ```bash
78      python -m backend.main --video-path "C:/path/to/my_video.mp4" --segmenter yolo_hybrid
79      python -m backend.main --video-path "C:/path/to/my_video.mp4" --segmenter gemini
80      python -m backend.main --video-path "C:/path/to/my_video.mp4" --segmenter motion
81      ```
82
83      ### 3. Rich CLI Debugging (`--debug-clip`)
84      To diagnose exactly why a video segment was registered as active/dead play, or inspect
detailed focus/continuity metrics at a particular time in seconds:
85      ```bash
86      python -m backend.main --video-path "C:/path/to/my_video.mp4" --debug-clip 125.5
87      ```
88      *Outputs granular validation check logs and motion ratios within a +/- 3 second frame
window around the timestamp.*
89
90      ### 4. Extracting an Arbitrary Trimmed Segment (`--trim-start` / `--trim-end`)
91      For fast iteration on a tiny slice of a multi-GB recording, you can stream-copy a
segment to disk without re-encoding:
92      ```bash
93      python -m backend.main --video-path "C:/path/to/my_video.mp4" \
94          --trim-start 600 --trim-end 660 --trim-output slice_10m_to_11m.mp4
95      ```
96      Output lands in `--output-dir` (default `output/`) unless `--trim-output` is an absolute
path.
97
98      ### 5. End-to-End "Process + Stitch" Script
99      `run_process_and_stitch.py` is an interactive convenience runner: it MD5-hashes a
hardcoded video path, reuses cached indexed segments if available (offering to re-index from
scratch), and then prompts for stitching parameters (`begin_padding`, `end_padding`,
`min_shot_count`) before compiling the final highlight reel. Useful when iterating on stitching
```

settings against a fixed large recording without re-running expensive segmentation.

?? Threshold Tuning (`config.json`)
You can adjust the parameters of your validators, segmenters, AI provider, and stitcher by editing `config.json` in the root folder. The default config emitted by `setup.py`:

```json
{
    "sport": "badminton",
    "ai_provider": "gemini",
    "ai_model": "gemini-2.5-flash",
    "validation": {
        "min_resolution_width": 1280,
        "min_resolution_height": 720,
        "min_fps": 29.9,
        "min_blur_threshold": 20.0,
        "max_scene_cuts_per_minute": 0.5,
        "relevance": {
            "court_green_lower": [35, 40, 40],
            "court_green_upper": [85, 255, 255],
            "court_pixels_min_ratio": 0.05
        }
    },
    "indexing": {
        "default_segmenter": "yolo_hybrid",
        "use_gpu": true,
        "motion_threshold": 0.08,
        "min_segment_duration": 3.0,
        "dead_time_merge_gap": 2.0,
        "chunk_duration": 90,
        "chunk_overlap": 10,
        "yolo_velocity_threshold": 0.15,
        "yolo_frame_skip": 3,
        "yolo_tracker": "bytetrack.yaml",
        "yolo_parallel_chunks": false,
        "yolo_chunk_workers": 2,
        "min_action_window_duration": 2.0,
        "ai_handoff_padding": 2.0,
        "ai_max_workers": 5
    },
    "output": {
        "default_highlights_name": "highlights.mp4",
        "db_name": "sports_indexer.db"
    },
    "stitching": {
        "begin_padding": 0.5,
        "end_padding": 0.5,
        "use_cache": false,
        "min_shot_count": 1
    }
}
```

Notable knobs:
*   `validation.min_blur_threshold` ? defaults to `20.0` (deliberately permissive for noisy indoor-arena GoPro footage; see `TEST_RUN_SUMMARY.md` for why).
*   `indexing.default_segmenter` ? `yolo_hybrid` | `gemini` | `motion`.
*   `indexing.use_gpu` ? when true and CUDA is available, YOLO runs on GPU.
*   `indexing.yolo_velocity_threshold` ? fraction of frame width per second; lower values catch slower play, higher values are stricter about what counts as "action".
*   `indexing.yolo_frame_skip` ? process every Nth frame during YOLO tracking. `3` at 30 fps ? 10 samples/sec.
*   `indexing.chunk_duration` / `chunk_overlap` ? how to split long videos for the YOLO

phase.
160   *   `indexing.ai_handoff_padding` ? seconds of slack added around each YOLO candidate window before sending to the AI provider.
161   *   `indexing.ai_max_workers` ? parallel calls to the AI provider during phase 3.
162   *   `stitching.min_shot_count` ? drop play segments whose estimated shot count is below this threshold when compiling highlights.
163
164   ---
165
166   ## ? Frequently Asked Questions (FAQ)
167
168   ### Q1: The server complains that `ffprobe` is not found.
169   Ensure FFmpeg is installed and added to your system environment variables. You can check this by typing `ffmpeg -version` or `ffprobe -version` in your terminal. If running `setup.py` outputs a `[FAIL]`, re-install FFmpeg or manually copy the binary files to a PATH folder.
170
171   ### Q2: Why does my video fail the Relevance check?
172   Our default relevance check scans for the typical indoor green badminton court floor color space (HSV). If your court is blue, or uses standard wood grain flooring, simply adjust the HSV values inside your `config.json` under `validation.relevance.court_green_lower` and `court_green_upper` to match your court, or lower the `court_pixels_min_ratio` to `0.0` to bypass.
173
174   ### Q3: How do I add a new custom validator to the pipeline?
175   1. Create a new python file in `backend/validators/` (e.g. `my_check.py`).
176   2. Subclass `VideoValidator` and implement `validate(self, video_path, config, context)`.
177   3. Register your class inside `backend/validators/__init__.py` using `registry.register(MyCheckValidator())`.
178   It's that simple! The app will automatically run it, display it in the CLI diagnostics summary, and render it in the frontend checklist.
179
180   ### Q4: Why does the CLI or server output freeze or take a long time on large videos?
181   *   **Print Buffering**: Standard Python buffers console output when stdout is redirected or running in background shells on Windows. To get real-time unbuffered log statements immediately, run commands with Python's unbuffered flag:
182   ```bash
183   python -u -m backend.main --video-path "C:/path/to/video.mp4"
184   ```
185   *   **Video Seeking Overhead**: Seeking to random frames in high-bitrate long H.265/HEVC videos using OpenCV's `cap.set(cv2.CAP_PROP_POS_FRAMES)` incurs massive I/O decoding delays. The `motion` segmenter reads sequentially and skips frame decodes using container-level `cap.grab()`, which delivers a **100x speedup** on multi-gigabyte raw files. `yolo_hybrid` sidesteps the problem entirely by extracting fixed chunks via ffmpeg first.
186
187   ### Q5: Why did the video fail with "No keyframes / packet timestamps could be extracted"?
188   Some H.265/HEVC streams (e.g. GoPro footage) encode frames as GDR or CRA keyframes. Standard FFmpeg frame-level skipping (`-skip_frame nokey`) fails to identify them, returning empty logs. We resolved this by extracting packet indexes (`packet=pts_time,flags`) at the container demuxer level instead, which is incredibly robust and executes instantly without decoding a single pixel.
189
190   ### Q6: How can I quickly test the indexing without waiting to parse an entire multi-GB video file?
191   You can pass the `--max-frames <N>` argument in CLI or `max_frames` in the POST API payload. This restricts the segmenter frame loop to process only the first N sub-sampled frames, enabling you to test motion filters and court relevance checks in seconds:
192   ```bash
193   python -m backend.main --video-path "C:\Users\avidu\Videos\LargeTestVideo.MP4" --max-frames 300
194   ```
195   *(At 5 sub-sampled frames per second, `--max-frames 300` analyzes exactly the first 60 seconds of video).*
196

Q7: How do I configure and run an AI-powered segmenter (Gemini / YOLO-hybrid)?

Both `yolo_hybrid` (default) and `gemini` need an API key for the AI provider. To use Google Gemini:

1. Create a local `.env` file in the project root (this is already in `.gitignore` to prevent secret leaks).
2. Set your Google AI Studio API key in the file:
   ```env
   GEMINI_API_KEY=your_key_here
   ```
3. Run with whichever segmenter you want:
   ```bash
   # Default: cheap candidate filtering with YOLO, then precise boundaries via Gemini
   python -m backend.main --video-path "C:/path/to/video.mp4" --segmenter yolo_hybrid

   # Full-video Gemini analysis (more expensive, highest semantic precision per call)
   python -m backend.main --video-path "C:/path/to/video.mp4" --segmenter gemini

   # Pure CV baseline ? no API key needed
   python -m backend.main --video-path "C:/path/to/video.mp4" --segmenter motion
   ```
   Or set `"default_segmenter": "yolo_hybrid"` (or `"gemini"`/`"motion"`) under `"indexing"` in `config.json`.

The AI provider used by `yolo_hybrid` and `gemini` is controlled by `ai_provider` + `ai_model` at the top level of `config.json`. Today only `"gemini"` is registered; new providers can be added under `backend/providers/`.

Q8: How does the `yolo_hybrid` pipeline actually save money?

Sending a multi-gigabyte video to a hosted LLM is slow and expensive. `yolo_hybrid` runs in four phases:

1. **Chunking** ? splits the video into overlapping windows (`chunk_duration`, `chunk_overlap`) so we can process incrementally.
2. **Detection + tracking** ? torchvision person detection (Faster R-CNN) + ByteTrack runs on each chunk locally (GPU-accelerated when available), measuring per-track velocity. Frames whose normalised player velocity exceeds `yolo_velocity_threshold` are kept; the rest are dropped. Adjacent active frames are merged into candidate windows, with sub-`min_action_window_duration` windows discarded.
3. **AI handoff** ? only the surviving candidate windows (padded by `ai_handoff_padding` seconds on each side) are extracted and sent to the AI provider for precise rally boundaries and shot-count estimation. These calls run in parallel up to `ai_max_workers`.
4. **DB population** ? confirmed segments are written to `play_segments` along with `shot_count`, `shot_count_confidence`, and a `detector` tag like `yolo-hybrid-gemini-2.5-flash`.

In practice this typically cuts AI-provider runtime/cost by an order of magnitude versus uploading the full video to `gemini` directly, while preserving the LLM's strength at semantic boundary detection.

Q9: How can I debug/test the Gemini Segmenter on a small portion of a video without uploading a huge multi-GB file?

Ingesting and uploading multi-gigabyte video files to Google Cloud can be slow and expensive. We built a native **fast local trimming** developer debugging feature:

1. Open your `config.json` file.
2. Under the `"indexing"` configuration block, add a `"debug_duration"` key (in seconds):
   ```json
   "indexing": {
       "default_segmenter": "gemini",
       "debug_duration": 15.0,
       ...
   }
   ```
3. When `"debug_duration"` is set, the Gemini segmenter will instantly run a zero-re-encode, sub-second `ffmpeg` slice to create a tiny temporary clip of the first 15 seconds, upload only that tiny clip to Google AI Studio (which takes <1 second to upload and <5 seconds

to process), run the model query, and clean up the temporary files automatically. This allows you to iterate on model prompts, evaluate validation overlaps, and verify rally boundaries in under 10 seconds!

Q10: Can I trust the per-segment metadata from the `motion` segmenter?

No. The zero-dependency `motion` segmenter detects *timing* via pixel-difference, but its per-segment **metadata is heuristic/synthetic** ? `intensity`, `avg_speed_kmh`, the serve/smash/foul/winner **events**, and `server`/`receiver` are generated values, **not measured**. They exist so the schema is populated for demos; do not treat them as ground truth. For real shot counts and boundaries use `yolo_hybrid` (default) or `gemini`. If your observability report shows a `synthetic_motion_metadata` warning, this is why.

Q11: I re-ran a video and my previous results changed/disappeared. Why?

Videos are keyed by the **MD5 of the file's bytes**. Re-processing the *same* file is treated as a fresh ingest: `add_video` is `INSERT OR REPLACE`, and the foreign-key cascade **deletes the previous play/candidate segments** before writing the new ones. So the earlier results are intentionally replaced, not merged. (A `reingest_replaced_prior` note in the report flags this.) Note also that re-encoding or re-downloading a video changes its bytes ? a new MD5 ? a separate record.

Q12: What is the `*.report.json` / `*.report.md` / `*.summary.md` file next to my video?

Every processing run now writes a **per-video observability report** next to the video (or to `report.output_dir` if set in `config.json`):

- **`<name>.report.json`** ? machine-readable: every stage, metric, and finding. The source of truth; diff two runs to see what changed.
- **`<name>.report.md`** ? the **technical** report: validation results, processing stages, and **findings/warnings**. Each warning names the problem and cites the FAQ entry to read. **Start here when a run looks wrong.**
- **`<name>.summary.md`** ? a **customer-friendly** summary: just the video metadata and highlights, no system internals.

To read the technical report: scan **Verdict** first (PASS / PASS-with-warnings / FAIL), then **Findings & warnings** ? each finding has a plain-English message, evidence, and a `? see README#Qn` pointer. To turn reports off or relocate them, set `"report": {"enabled": false}` or `"report": {"output_dir": "..."}` in `config.json`.

Q13: A validation check failed and indexing was halted ? what do I do?

Open the report's **Processing stages ? validation** block (or the `VALIDATION SUMMARY` printed to the console). Each failed check states the exact reason. Common cases: **relevance** (the court color didn't match ? see [Q2](#q2-why-does-my-video-fail-the-relevance-check)); **resolution/fps** (below the configured minimums ? adjust `validation.min_*` in `config.json`); **format** (not an MP4 container); **keyframes/PTS** (HEVC/GoPro keyframe quirks ? see [Q5](#q5-why-did-the-video-fail-with-no-keyframes--packet-timestamps-could-be-extracted)). Fix the underlying cause or relax the relevant threshold in `config.json`, then re-run.

? Annotation & Evaluation

Once a video is processed, you can measure **how well the pipeline detected its rallies** against hand-annotated ground truth. You annotate each rally's `[start, end)` time range in a small CSV; the harness scores the detected rallies straight from the SQLite DB ? no API calls, fully deterministic.

```bash
python run_eval.py --scaffold --annotations rallies.csv          # make an empty template
# ...fill in start,end rows, then:
python run_eval.py --video "C:/videos/match.mp4" --annotations rallies.csv --show-failures
```

It reports precision/recall/F1, mean IoU, and boundary error for **two stages** ? raw detector `candidates` vs. AI-confirmed `final` segments ? which pinpoints whether a weak result

is a *detection* problem or a *segmentation* problem.

> Rally timestamps evaluate and tune **segmentation**; they do **not** train the shuttle detector (that needs per-frame coordinate labels). See **docs/EVALUATION.md** for the annotation format, CLI flags, and how to read the output.

? Running the Test Suite

The test suite uses `pytest` and lives in `tests/`. From the project root:

```bash
pip install pytest pytest-cov --user
pytest                              # run everything
pytest tests/test_yolo_hybrid.py    # run a single file
pytest -k yolo --maxfail=1 -v       # filter by keyword, fail fast, verbose
pytest --cov=backend --cov-report=term-missing   # coverage report (uses .coveragerc)
```

The suite mocks external dependencies (the Gemini client, ffprobe/ffmpeg, OpenCV `VideoCapture`, the YOLO model) so it runs offline and without a GPU. The tests cover:

* `tests/test_base.py` ? segmenter registry
* `tests/test_database.py` ? SQLite schema, video/segment insert/query
* `tests/test_geometry.py` ? bbox center / distance / velocity / IoU helpers
* `tests/test_video.py` ? ffprobe duration + ffmpeg chunk extraction helpers
* `tests/test_providers.py` + `tests/test_provider_gemini.py` ? AI provider registry and the Gemini provider
* `tests/test_sport_profiles.py` ? sport profile registry + the badminton prompt
* `tests/test_gemini.py` ? `GeminiPlaySegmenter` (single-file and chunked paths)
* `tests/test_yolo_hybrid.py` ? `HybridYoloSegmenter` happy path, missing-API-key bailout, and timestamp-validation drops
* `tests/test_tracknet_runner.py` + `tests/test_tracknet_hybrid.py` ? TrackNet WSL runner (mocked subprocess) and the `tracknet_hybrid` segmenter
* `tests/test_eval_metrics.py` + `tests/test_eval_annotations.py` + `tests/test_eval_harness.py` ? rally evaluation harness (interval matching, CSV loading, DB scoring, CLI)

? Project Structure (high level)

```
backend/
  main.py                # FastAPI app + CLI orchestrator
  database.py            # SQLite schema and queries
  validators/            # Pluggable video validators (format, blur, scene-cut, relevance, ...)
  indexer/               # Pluggable rally segmenters: motion, gemini, yolo_hybrid, tracknet_hybrid
  detectors/             # TrackNet WSL2 runner (shuttle-trajectory subprocess adapter)
  providers/             # Pluggable AI providers (gemini today)
  sports/                # Pluggable sport profiles (badminton today)
  eval/                  # Rally evaluation harness: metrics, annotations, scoring
  stitcher/compiler.py   # FFmpeg-based highlight compiler
  utils/                 # ffprobe/ffmpeg + bbox geometry helpers
  static/                # Zero-build frontend served by FastAPI
docs/                    # DECISIONS (ADRs), TRACKNET_WSL_SETUP, EVALUATION
tests/                   # pytest suite (mocked, runs offline)
run_process_and_stitch.py # Interactive end-to-end runner
run_eval.py              # Rally evaluation CLI (see "Annotation & Evaluation" above)
setup.py                 # Dependency & FFmpeg diagnostics + default config init
config.json              # Runtime configuration (see "Threshold Tuning" above)
```

325

4. user (2026-06-07T08:14:00.325Z)

```
1       # Badminton Highlight Indexer ? Video Report
2
3       ## Run
4
5       | Field | Value |
6       | --- | --- |
7       | Tool | badminton-highlight-indexer |
8       | Video id | md5a (file_md5) |
9       | Video path | /videos/match.mp4 |
10      | Generated | 2026-06-07T08:10:50Z |
11      | Schema | 1.0 |
12
13      ## Video
14
15      | Field | Value |
16      | --- | --- |
17      | Duration | 11m16s |
18      | Resolution | 1920x1080 |
19      | FPS | 59.94 fps (probed) |
20      | Container | mov,mp4,m4a |
21
22      ## Processing stages
23
24      **validation** ? `ok` (0.0s)
25        - checks_passed: 2
26        - checks_total: 2
27
28      **segmentation** ? `ok` (0.0s)
29        - candidate_windows: 1
30        - segment_count: 0
31        - total_play_s: 0 s
32        - _segmenter_requested_: wasb_hybrid
33        - _segmenter_actual_: wasb_hybrid
34        - _config_default_: wasb_hybrid
35        - _segmenter_explicitly_requested_: True
36        - _matches_config_default_: True
37        - _was_reingest_: False
38        - _detector_: None
39
40      **ai_handoff** ? `ok` (0.0s)
41        - confirmed_segments: 0
42        - _ai_based_: True
43        - _provider_: gemini
44        - _model_: gemini-flash
45        - _api_key_present_: False
46
47      ## Findings & warnings
48
49      - [ERROR] **silent_provider_noop** ? 0 segments AND no AI provider API key was set ?
this is almost certainly a silent provider no-op, not 'the video has no rallies'. Set the
provider key (e.g. GEMINI_API_KEY in a .env file) and re-run.
50        ? see **README#Q7**
51        ? evidence: stages.ai_handoff.details.api_key_present=False,
stages.segmentation.metrics.segment_count=0
52
53      ## Verdict
54
55      **FAIL**
56
```

5. assistant (2026-06-07T08:14:11.353Z)

Based on reading exactly those two files, here is my diagnosis:

Diagnosis

The tool attempted to run the segmenter `wasb_hybrid` (which appears to be a typo or misconfiguration ? the README mentions `yolo_hybrid`, not `wasb_hybrid`). During the segmentation phase, the AI handoff stage detected that **no GEMINI API key was present** (`_api_key_present_: False`), yet the tool proceeded silently without processing any actual segments. This resulted in 0 confirmed segments being written to the database, making it appear as if "the video has no rallies" ? when in reality, the segmenter never even ran because a required credential was missing.

The report correctly flags this as an **ERROR** (`silent_provider_noop`) with the plain-English message: "0 segments AND no AI provider API key was set ? this is almost certainly a silent provider no-op, not 'the video has no rallies'."

Exact Fix

1. **Create a `.env` file** in the project root directory (C:\Users\avidu\Projects\badminton-highlight-indexer\) containing:
   ```env
   GEMINI_API_KEY=your_actual_google_ai_studio_key_here
   ```
2. **Obtain a key** from Google AI Studio if you don't have one.
3. **Verify the segmenter name** in `config.json` is set to a valid value: either `yolo_hybrid` (default), `gemini`, or `motion`. The report shows `wasb_hybrid`, which is not recognized ? this should be corrected.
4. **Re-run the processing command** with the corrected segmenter and API key in place.

FAQ Section Used

README#Q7 ? "How do I configure and run an AI-powered segmenter (Gemini / YOLO-hybrid)?" ? specifically the instruction to create a `.env` file with `GEMINI_API_KEY=your_key_here`.

Self-Debug Assessment

Yes, I could self-debug this from the report alone. The report is exceptionally clear:
- The findings section explicitly names the problem ("silent provider no-op")
- It directly cites which FAQ to read (README#Q7)
- The evidence fields pinpoint the root cause: `api_key_present=False` combined with `segment_count=0`

Confusing or Ambiguous Parts

1. **The segmenter name `wasb_hybrid`** ? This is not mentioned anywhere in the README. It appears to be a typo or misconfiguration, but the report doesn't explain what happened or suggest how to correct it. Users might assume this is a valid option and get stuck wondering what `wasb_hybrid` is.

2. **"Silent provider no-op"** ? The term "no-op" (no-operation) is jargon. A clearer phrasing for non-technical users might be: "The AI provider was supposed to run but did nothing because the API key was missing."

3. **No clarification on which `.env` location is expected** ? While Q7 says "in the project root," it would help to state this even more explicitly in the error message itself (e.g., "Create `.env` in C:\Users\avidu\Projects\badminton-highlight-indexer\").

---

6. user (2026-06-07T08:14:14.381Z)

Structured output provided successfully